EXTERNAL BRAIN

Give your AI a memory that survives the session.

Your coding tools forget everything between sessions. This wires them to a Brain that remembers what you decided and why — then hands it back before you have to re-explain it. Three steps.

5 min to working | 10 clients supported | One command

1

Token

Mint a token

Sign in and open **Settings → Tokens**, then **Create token**.

Name it after the *machine*, not yourself — `laptop`, `work-desktop`, `ci-runner`. You revoke per machine, so per-machine names are what make revocation useful later.

> **The token is shown once.** Only its SHA-256 hash is stored — there is no reveal button and no recovery. If you lose it, mint a new one and revoke the old.

The screen then shows an install command with your token already in it. That command is Step 2 — copy it and skip ahead.

2

Connect

Connect your AI tool

One command, whichever tool you use. `--client` defaults to `claude-code`.

MACOS · LINUX · WSL · GIT BASH

```
curl -fsSL https://<your-brain>/api/onboard.sh \
  | bash -s 'bp_…' --client claude-code
```

WINDOWS POWERSHELL 5.1+

```
iwr https://<your-brain>/api/onboard.ps1 -UseBasicParsing | iex
Install-Brain -Token 'bp_…' -Client claude-code
```

PICK YOUR CLIENT

YOUR TOOL	FLAG	WORTH KNOWING
Claude Code	`claude-code`	Also installs the Brain skill
Claude Desktop	`claude-desktop`	Needs Node. **Fully quit** the app after — closing the window isn't enough
Cursor	`cursor`	
Windsurf	`windsurf`	
Google Antigravity	`antigravity`	One config serves both the IDE and the CLI
VS Code + Copilot	`vscode`	Run it from your repo root — writes `./.vscode/mcp.json`
GitHub Copilot CLI	`copilot-cli`	
OpenAI Codex	`codex`	Also prints an `export BRAIN_TOKEN=…` line. Add it to your shell profile or Codex gets a 401
Anything else	`generic`	Add `--config-path <file>` and it writes that file for you

JetBrains, Visual Studio, Eclipse and Xcode have no stable config path, so paste the JSON from the mint screen instead.

WHAT THAT COMMAND ACTUALLY DOES

1. **Writes the config** — through your tool's own `mcp add` where one exists, otherwise by *merging* into its config file. Your other MCP servers survive, and the file is backed up first.
2. **Proves it works** — a real connection and tool call through your network and auth. This is why it can claim success honestly: a config file being written proves nothing about whether the token can actually call anything.

3. **Logs a first session**, so your dashboard isn't empty on day one.

> **Then restart your AI tool.** Every one of them reads MCP config only at startup. This is the single most common cause of “I installed it and nothing happened”.

RATHER READ IT BEFORE PIPING TO A SHELL?

AUDIT FIRST

```
curl -fsSL https://<your-brain>/api/onboard.sh -o /tmp/brain-inst
less /tmp/brain-install.sh
bash /tmp/brain-install.sh 'bp_…' --client cursor
```

3

Talk

Talk to it like a colleague

You never call tools by name. You speak normally, and the installed skill tells your AI which tool to reach for.

YOU SAY…	WHAT HAPPENS
(you start any real task)	A session opens; past rules that match get handed to the model before it writes anything
“remember this: we use pgvector, not Pinecone”	Saved as a rule — it shows up under Skills
“ask the oracle: how did we solve the migration ordering?”	A synthesised answer, with citations back to the rules it used
“find the rule about prisma migrations”	Closest matches by meaning. Instant and free — no model call
“what did I do last week on billing?”	Past sessions matching those words
“that worked” · “we’re done” · “ship it”	**Closes the session — this is what makes it learn**

THE ONE HABIT THAT MATTERS

Say “that worked” when you finish.

An unclosed session teaches the Brain nothing — extraction only fires on close. This is the number one reason a Brain sits empty after a week of real use.

Everything else here is optional. This isn't.

A SESSION, START TO FINISH

WHAT IT LOOKS LIKE

```
You     I need to refactor the auth middleware to use the new
        credentials adapter.

Claude  [opens a session, retrieves matching rules]
        Two of your rules apply here: sessions bind at start,
        and credential checks belong at the repository layer.

You     no — we moved that check to the route handler last month.

Claude  Noted, updating.

You     that worked, we're done.

Claude  [closes the session with what it learned]
        Saved: "credential checks live at the route handler,
        not the repository layer — moved 2026-07."
```

That correction is now permanent. Next month, on a different machine, it arrives before you have to remember it.

GETTING GOOD ANSWERS

- **“find the rule about X”** — instant, free, no model call. Use it any time you'd otherwise re-derive a past decision.
- **“ask the oracle: …”** — slower and billed, but synthesised with citations. Use it when you want reasoning, not a list.
- **Don't** ask the Oracle general programming questions. It's your project's memory, not the internet. “How does flexbox work” is a web search; “how did *we* decide to lay out the dashboard” is the Oracle.

✓
Check

Confirm it's live

VERIFY

```
claude mcp list | grep brain
# brain: https://…/mcp (HTTP) - ✓ Connected
```

Then ask it, in your tool: *“ask the brain what it knows about this project”*. If it answers at all, the loop is live.

IF SOMETHING'S OFF

SYMPTOM	ALMOST ALWAYS
Your tool doesn't see the Brain	It wasn't restarted after installing
401 on every call	Token revoked, expired, or minted on a different Brain
Codex specifically 401s	`BRAIN_TOKEN` missing from your shell profile
Connected, but Skills stays empty	Sessions never closed — say “we're done”
Edited `~/.claude/mcp.json`, nothing changed	Wrong file. Claude Code reads `~/.claude.json`

WHERE NEXT

Getting started
The same ground, slower, with diagrams

Asking the Oracle
Question patterns that work

Teaching knowledge

Managing tokens

Writing rules that survive

Scope, rotation, revocation

Troubleshooting

When it isn't helping

Client reference

Per-tool config shapes and traps
